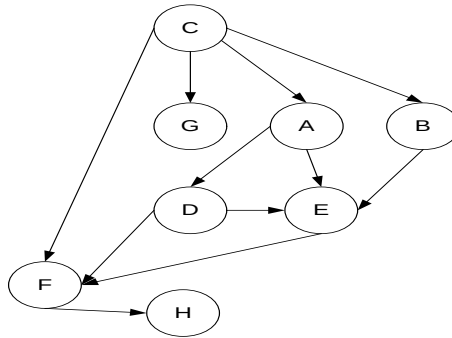


GUJARAT TECHNOLOGICAL UNIVERSITY**BE - SEMESTER-V EXAMINATION – SUMMER 2025****Subject Code:3150703****Date:28-05-2025****Subject Name:Analysis and Design of Algorithms****Time:02:30 PM TO 05:00 PM****Total Marks:70****Instructions:**

1. Attempt all questions.
2. Make suitable assumptions wherever necessary.
3. Figures to the right indicate full marks.
4. Simple and non-programmable scientific calculators are allowed.

		MARKS
Q.1	(a) Find Big-oh (O) notation of following functions $f(n)=3n^6 + 2n \cdot \lg n + 6n$, $h(n)= 3n^2+ 6n \cdot \lg n+ 2n^{2.5}$	03
	(b) Define Omega and Theta notations with graph.	04
	(c) Write binary search algorithm and analyze it for worst case time complexity. Represent its time complexity using Big-oh (O) notation.	07
Q.2	(a) Prove that $(n + 6)^3 = \Theta(n^3)$.	03
	(b) If $P(n) = a_0 + a_1 n + a_2 n^2 + \dots + a_m n^m$ then prove that $P(n) = \Omega(n^m)$. Here $a_0, a_1, a_2, \dots, a_m$ are constants and $a_m > 0$.	04
	(c) Solve following recurrence relation using suitable method and express your answer using Big-oh (O) notation. $T(n) = T(n/6) + T(5n/6) + \Theta(n)$	07
	OR	
	(c) Solve following recurrence relations using suitable method and express your answer using Big-oh (O) notation. 1. $T(n) = 2 T(n/2) + n \log n$ 2. $T(n) = 4 T(n/4) + n$	07
Q.3	(a) Arrange the following growth rates from the lowest to highest asymptotic order: $O(n \log n)$, $O(n^{2n})$, $\Omega(\log n)$, $O(n^{0.5})$, $O(n!)$, $\Omega(2^n)$, $O(n^{0.5} \log n)$	03
	(b) Write the recurrence relation for the quick sort on input instance: 15, 19, 20, 25, 32, 37, 50, 62, 70. Comment on the nature of input i.e. best case, average case or worst case.	04
	(c) Write greedy algorithm for activity selection problem. Give its time complexity. For following intervals, select the activities according to your algorithm. I_1 (1-3), I_2 (0-2), I_3 (3-6), I_4 (2-5), I_5 (5-8), I_6 (3-10), I_7 (7-9).	07
	OR	
Q.3	(a) Prove that $\text{Log}(\sqrt{n}) = O(\log n)$.	03
	(b) Write any algorithm (need not to have meaningful) which has a time complexity of $O(1)$ and also includes a loop statement. Also, confirm its time complexity of $O(1)$ using tabular method.	04

- (c) Write DFS traversal and draw DFS tree corresponds to following graph. Also, write time complexity of DFS algorithm. 07



- Q.4** (a) What is Principle of Optimality in dynamic programming? Explain it with example. 03
- (b) What is knapsack problem? Using greedy algorithm find an optimal solution for fractional knapsack instance $n=7$, $M=17$, profits $p[] = \{10, 5, 15, 7, 6, 16, 4\}$ and weights $w[] = \{2, 3, 5, 7, 1, 4, 1\}$. 04
- (c) Find the optimal way of multiplying following matrices using dynamic programming. Also indicate optimal number of multiplications required. 07
 $A: 2 \times 3$, $B: 3 \times 5$, $C: 5 \times 6$, $D: 6 \times 2$, $E: 2 \times 3$

OR

- Q.4** (a) What is the Travelling Salesman Problem (TSP)? Why TSP is considered an NP-hard problem? 03
- (b) Illustrate the working of Kruskal's algorithm using greedy technique. Take the suitable graph for illustration. 04
- (c) Find the longest common subsequence for the following two sequences using dynamic programming. Show the complete process and indicate the time complexity. 07
 $X = 10101010011$
 $Y = 101001$

- Q.5** (a) Define P and NP problems. Also give example of each type of problem. 03
- (b) Explain the backtracking strategy to solve optimization problem. Explain how it can be applied to find the solution for 4-queen problem. 04
- (c) Given a text and a pattern, write the naive string matching algorithm to find all occurrences of the pattern in the text. Use the text "ABCABCABC" and the pattern "ABC". Also, give time complexity of naive string matching algorithm. 07

OR

- Q.5** (a) What is a polynomial reduction? Explain its role in proving NP-completeness. 03
- (b) Give example of any NP complete problem. Explain how to prove that particular problem is NP complete. 04
- (c) Show that Hamiltonian cycle is a NP problem. 07

Q.1 (a) Find Big-oh (O) notation of following functions $f(n)=3n^6 + 2n \cdot \lg n + 6n$, $h(n)= 3n^2+ 6n \cdot \lg n+ 2n^{2.5}$ (3 mark)

Function 1:

$$f(n) = 3n^6 + 2n \cdot \log n + 6n$$

Solution:

Big-Oh notation represents the **upper bound** of the growth rate of a function and focuses on the **dominant term** for large values of n .

In the given function:

- $3n^6$ is a polynomial term of degree 6
- $2n \log n$ grows slower than n^6
- $6n$ is a linear term and grows much slower

As $n \rightarrow \infty$, the term $3n^6$ dominates all other terms.

Constant coefficients are ignored in Big-Oh analysis.

Therefore:

$$f(n) = O(n^6)$$

Function 2:

Solution:

To find Big-Oh, compare the growth rates of each term:

- $3n^2$ is a quadratic term
- $6n \log n$ grows faster than n but slower than n^2
- $2n^{2.5}$ is a polynomial term with power 2.5

Among all terms, $n^{2.5}$ grows the fastest for large values of n .
Lower-order terms and constant factors are ignored.

Therefore:

$$h(n) = O(n^{2.5})$$

Final Answers:

☐ $f(n) = O(n^6)$

☐ $h(n) = O(n^{2.5})$

(b) Define Omega and Theta notations with graph. (4 mark)

Omega (Ω) Notation

Definition:

Omega (Ω) notation is used to represent the **lower bound** of an algorithm's time complexity. It describes the **best-case performance** or the minimum time required by an algorithm to complete execution for large input size n .

Explanation:

An algorithm is said to be **$\Omega(g(n))$** if there exists a positive constant c and a value n_0 such that for all $n \geq n_0$, the running time $T(n)$ of the algorithm is **greater than or equal to** $c \cdot g(n)$.

This means the algorithm will **never run faster** than this bound.

Graph Explanation:

In the graph, the input size n is shown on the X-axis and time complexity on the Y-axis.

The curve representing $c \cdot g(n)$ lies **below** the curve of $T(n)$.

From a certain point n_0 , the actual running time always stays **above** the Ω curve, showing the minimum guaranteed performance.

Use of Omega Notation:

- To analyze best-case time complexity
 - To show minimum execution time
 - To compare algorithms based on lower bound
-

Theta (Θ) Notation

Definition:

Theta (Θ) notation represents the **tight bound** of an algorithm's time complexity. It defines both the **upper and lower bound**, giving the exact growth rate of the algorithm.

Explanation:

An algorithm is said to be $\Theta(g(n))$ if there exist positive constants c_1 , c_2 , and n_0 such that for all $n \geq n_0$:

This indicates that the algorithm's running time grows **at the same rate** as $g(n)$.

Graph Explanation:

In the graph, two curves $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ form **lower and upper bounds**.

The actual time complexity curve $T(n)$ lies **between these two lines** after n_0 , showing a tight bound.

Use of Theta Notation:

- To represent exact time complexity
 - To analyze average performance
 - To compare algorithms accurately
-

(c) Write binary search algorithm and analyze it for worst case time complexity. Represent its time complexity using Big-oh (O) notation. (7 mark)

Introduction / Definition

Binary Search is an **efficient searching algorithm** used to find a specific element in a **sorted array or list**. It works on the principle of **divide and conquer**, where the search space is repeatedly divided into two halves. Binary search significantly reduces the number of comparisons compared to linear search, making it suitable for large data sets.

Precondition of Binary Search

- The list or array **must be sorted** (either in ascending or descending order).
 - Random access to elements should be possible, which is why it is commonly applied on arrays.
-

Binary Search Algorithm

The algorithm works by comparing the target value with the **middle element** of the array.

Algorithm: BinarySearch(A, n, key)

- Step 1: Set $low = 0$ and $high = n - 1$
- Step 2: While $low \leq high$, repeat the following steps:
 - Calculate $mid = (low + high) / 2$
 - If $A[mid] == key$, then element is found and return index
 - If $key < A[mid]$, then set $high = mid - 1$

- Else set $low = mid + 1$
- Step 3: If $low > high$, then element is not present in the list

This process continues until the element is found or the search space becomes empty.

Working Explanation

Initially, the entire array is considered for searching. The algorithm checks the middle element and decides whether the required element lies in the **left half or right half**. Each comparison eliminates **half of the remaining elements**. This repeated halving of the search space makes binary search very efficient.

If we imagine a diagram, it would show a sorted array where the middle element is checked first, then the remaining half is chosen repeatedly until the element is found.

Worst Case Time Complexity Analysis

The **worst case** occurs when:

- The element is **not present** in the array, or
- The element is found only after reducing the array to a single element

At each step, the array size is reduced by half:

- After 1 comparison $\rightarrow n / 2$
- After 2 comparisons $\rightarrow n / 4$
- After k comparisons $\rightarrow n / 2^k$

The process stops when:

$$\frac{n}{2^k} = 1$$

Solving this:

Thus, the maximum number of comparisons required in the worst case is **$\log_2 n$** .

Big-Oh (O) Representation

Since constants and lower-order terms are ignored in asymptotic analysis, the worst case time complexity of binary search is:

$$\boxed{O(\log n)}$$

Advantages of Binary Search

- Very fast compared to linear search
 - Efficient for large datasets
 - Reduces number of comparisons drastically
-

Applications

- Searching elements in sorted arrays
- Database indexing
- Searching in dictionaries and phone directories

Q.2 (a) Prove that $(n + 6)^3 = \Theta(n^3)$. (3 mark)

Proof:

Theta (Θ) notation represents the **tight bound** of a function, meaning it gives both the **upper and lower bound** of the growth rate.

To prove $(n + 6)^3 = \Theta(n^3)$, we expand the given expression.

$$(n + 6)^3 = n^3 + 18n^2 + 108n + 216$$

For large values of n , the highest power term n^3 dominates the growth of the function, while lower-order terms such as n^2 , n , and constant terms become insignificant.

Now consider positive constants $c_1 = 1$, $c_2 = 2$, and $n_0 = 6$.

For all $n \geq n_0$:

$$n^3 \leq (n + 6)^3 \leq 2n^3$$

Thus, the function $(n+6)^3$ is bounded both above and below by constant multiples of n^3 .

Hence, it is proved that:

$$\boxed{(n + 6)^3 = \Theta(n^3)}$$

This shows that $(n+6)^3$ and n^3 grow at the same rate asymptotically.

(b) If $P(n) = a_0 + a_1 n + a_2 n^2 + \dots + a_m n^m$ then prove that $P(n) = \Omega(n^m)$. Here $a_0, a_1, a_2, \dots, a_m$ are constants and $a_m > 0$. (4 mark)

Given a polynomial function:

$$P(n) = a_0 + a_1 n + a_2 n^2 + \dots + a_m n^m$$

where $a_0, a_1, a_2, \dots, a_m$ are constants and $a_m > 0$.

For large values of n , the **highest degree term** $a_m n^m$ dominates the growth of the polynomial, while all lower-degree terms such as $n^{m-1}, n^{m-2}, \dots$ and constant terms become insignificant.

Since all terms are non-negative for sufficiently large n , we can write:

$$P(n) \geq a_m n^m$$

Let $c = a_m$ and choose a suitable n_0 such that for all $n \geq n_0$, the inequality holds true.

By the definition of Omega (Ω) notation:

$$P(n) \geq c \cdot n^m \text{ for all } n \geq n_0$$

Hence, it is proved that:

$$\boxed{P(n) = \Omega(n^m)}$$

This shows that the polynomial function grows at least as fast as n^m for large input sizes.

ChatGPT can make mistakes. Check important info. See Cookie Prefer

(c) Solve following recurrence relation using suitable method and express your answer using Big-oh (O) notation. $T(n) = T(n/6) + T(5n/6) + \Theta(n)$ (7 mark)

Introduction

A recurrence relation expresses the running time of a recursive algorithm in terms of its subproblems. To analyze such relations, methods like **Substitution method**, **Recursion Tree method**, and **Master Theorem** are used.

In this question, the given recurrence is **not directly solvable using**

Master Theorem because the subproblems are of **unequal sizes**.
Therefore, the **Recursion Tree method** is the most suitable approach.

Given Recurrence Relation

$$T(n) = T\left(\frac{n}{6}\right) + T\left(\frac{5n}{6}\right) + \Theta(n)$$

Here:

- The problem is divided into two subproblems of sizes $\frac{n}{6}$ and $\frac{5n}{6}$
 - The non-recursive work done at each level is proportional to n
-

Recursion Tree Method Explanation

In the recursion tree method, we expand the recurrence into levels and calculate the cost at each level.

Level 0 (Root Level):

- Problem size = n
- Cost at this level = $\Theta(n)$

Level 1:

- Subproblems:
 - $T(n/6)$
 - $T(5n/6)$
- Total work at this level:

$$\Theta\left(\frac{n}{6}\right) + \Theta\left(\frac{5n}{6}\right) = \Theta(n)$$

Level 2:

Each subproblem again divides similarly.

The sum of sizes of all subproblems at this level is still proportional to n .

So,

Total work at every level = $\Theta(n)$

Height of the Recursion Tree

The recursion continues until the problem size reduces to 1.

The **largest subproblem** at each level is $\frac{5n}{6}$, so the height of the tree is determined by:

$$\left(\frac{5}{6}\right)^k n = 1$$

Solving for k :

$$k = \log_{6/5} n = O(\log n)$$

Thus, the recursion tree has **$O(\log n)$ levels**.

Total Cost Calculation

- Cost per level = $\Theta(n)$
- Number of levels = $O(\log n)$

$$T(n) = \Theta(n) \times O(\log n)$$

Final Time Complexity (Big-Oh Notation)

Ignoring constants and lower-order terms, the total time complexity is:

$$T(n) = O(n \log n)$$

OR

(c) Solve following recurrence relations using suitable method and express your answer using Big-oh (O) notation. 1. $T(n) = 2T(n/2) + n \log n$ 2. $T(n) = 4T(n/4) + n$ (7 mark)

1. $T(n) = 2T(n/2) + n \log n$

Introduction

The given recurrence relation represents a **divide and conquer algorithm** where a problem of size n is divided into **2 subproblems**, each of size $n/2$. The additional work done outside the recursive calls is proportional to $n \log n$.

To solve this recurrence, the **Master Theorem** is the most suitable method.

Step 1: Identify parameters of Master Theorem

The standard form of Master Theorem is:

$$T(n) = aT(n/b) + f(n)$$

Comparing:

- $a = 2$
- $b = 2$
-

Step 2: Compute $n^{\log_b a}$

$$n^{\log_2 2} = n^1 = n$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$

$$f(n) = n \log n$$

Here,

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

This matches **Case 2 of Master Theorem**, where:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Final Solution for Recurrence 1

$$T(n) = O(n \log^2 n)$$

Conclusion (Recurrence 1)

The recurrence grows faster than linear but slower than quadratic. The extra logarithmic factor arises due to the $n \log n$ work at each level of recursion.

2. $T(n) = 4T(n/4) + n$

Introduction

In this recurrence, the problem is divided into **4 equal subproblems**, each of size $n/4$. The non-recursive work done at each level is linear. Again, the **Master Theorem** is suitable for solving this recurrence.

Step 1: Identify parameters

Comparing with the standard form:

$$T(n) = aT(n/b) + f(n)$$

- - $b = 4$
 - $f(n) = n$
-

Step 2: Compute $n^{\log_b a}$

$$n^{\log_4 4} = n^1 = n$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$

$$f(n) = n = \Theta(n^{\log_b a})$$

This corresponds to **Case 2 of Master Theorem**.

Final Solution for Recurrence 2

$$\boxed{T(n) = O(n \log n)}$$

Conclusion (Recurrence 2)

Since the work done at each level is linear and the height of the recursion tree is logarithmic, the overall time complexity becomes $O(n \log n)$.

Final Answers (Big-Oh Notation)

1. $T(n) = O(n \log^2 n)$

2. $T(n) = O(n \log n)$

Q.3 (a) Arrange the following growth rates from the lowest to highest asymptotic order: $O(n \log n)$, $O(n^{2n})$, $\Omega(\log n)$, $O(n^{0.5})$, $O(n!)$, $\Omega(2n)$, $O(n^{0.5} \log n)$ (3 mark)

Given growth rates:

$O(n \log n)$, O

(n^{2n}) , Ω

Introduction

Asymptotic growth rates are used to compare algorithms based on how their time or space requirements increase with input size n .

While arranging growth rates, we consider how fast each function grows as n becomes very large, ignoring constants and lower-order terms.

Step-by-Step Comparison

- $\Omega(\log n)$ grows very slowly and is the smallest among all given functions.

- $O(n^{0.5})$ grows faster than logarithmic but slower than linear functions.
- $O(n^{0.5} \log n)$ grows slightly faster than $n^{0.5}$ due to the logarithmic factor.
- $O(n \log n)$ grows faster than all sub-linear functions but slower than exponential and factorial functions.
- $\Omega(2^n)$ is an exponential function and grows much faster than any polynomial function.
- $O(n^{2^n})$ is a very fast-growing exponential-type function and grows faster than 2^n .
- $O(n!)$ grows the fastest among all given functions and dominates all polynomial and exponential functions.

Final Arrangement (Lowest to Highest Asymptotic Order)

$\Omega(\log n) < O(n^{0.5}) < O(n^{0.5} \log n) < O(n \log n) < \Omega(2^n) < O(n^{2^n}) < O(n!)$
--

(b) Write the recurrence relation for the quick sort on input instance: 15, 19, 20, 25, 32, 37, 50, 62, 70. Comment on the nature of input i.e. best case, average case or worst case. (4 mark)

Given input instance:

15, 19, 20, 25, 32, 37, 50, 62, 70

Introduction

Quick Sort is a **divide and conquer sorting algorithm** that works by selecting a **pivot element**, partitioning the array around the pivot, and then recursively sorting the left and right subarrays. The efficiency of Quick Sort mainly depends on how the pivot divides the input array.

Analysis of Given Input

The given input sequence is already **sorted in ascending order**.

In standard Quick Sort implementation, if the **first or last element** is chosen as the pivot, then:

- One subarray will contain **$n - 1$ elements**
- The other subarray will contain **0 elements**

Thus, the partitioning becomes highly unbalanced.

Recurrence Relation

For this input, at each step:

- One recursive call is on a subarray of size $n - 1$
- The other recursive call is on a subarray of size 0
- Partitioning cost at each level is proportional to n

So, the recurrence relation is:

$$T(n) = T(n - 1) + T(0) + \Theta(n)$$

Since $T(0)$ is constant, it can be ignored:

$$T(n) = T(n - 1) + \Theta(n)$$

Nature of Input

This type of recurrence occurs when Quick Sort receives an **already sorted input** and uses a poor pivot selection strategy (first or last element as pivot).

Therefore, the given input represents the:

Worst Case of Quick Sort

(c) Write greedy algorithm for activity selection problem. Give its time complexity. For following intervals, select the activities according to your algorithm. I1 (1-3), I2 (0-2), I3 (3-6), I4 (2-5), I5 (5-8), I6 (3-10), I7 (7-9). (7 mark)

Introduction / Definition

The **Activity Selection Problem** is a classic problem that can be solved using the **Greedy method**. The objective of this problem is to select the **maximum number of non-overlapping activities** from a given set of activities, where each activity has a **start time** and a **finish time**.

The greedy strategy used here is to always select the activity that **finishes earliest**, leaving maximum time for the remaining activities.

Greedy Strategy Used

- Sort all activities in **ascending order of their finish times**
- Always select the activity with the **earliest finish time**
- Then select the next activity whose start time is **greater than or equal to** the finish time of the previously selected activity

This approach works optimally because it maximizes the remaining time for future activities.

Greedy Algorithm for Activity Selection

Algorithm: Activity_Selection(A)

- Step 1: Sort all activities according to their **finish time**
- Step 2: Select the first activity (earliest finish time)

- Step 3: For each next activity:
 - If its start time $\geq$ finish time of last selected activity
→ Select this activity
- Step 4: Repeat until all activities are considered

This algorithm ensures that the maximum number of mutually compatible activities are selected.

Given Activities

Activity Interval

I1	(1 – 3)
I2	(0 – 2)
I3	(3 – 6)
I4	(2 – 5)
I5	(5 – 8)
I6	(3 – 10)
I7	(7 – 9)

Step 1: Sort Activities by Finish Time

After sorting based on finishing time:

- I2 (0 – 2)
- I1 (1 – 3)
- I4 (2 – 5)
- I3 (3 – 6)

- I5 (5 – 8)
- I7 (7 – 9)
- I6 (3 – 10)

Step 2: Activity Selection Process

- Select **I2 (0 – 2)** → finishes earliest
- I1 (1 – 3) → overlaps with I2 → reject
- Select **I4 (2 – 5)** → start ≥ 2
- I3 (3 – 6) → overlaps with I4 → reject
- Select **I5 (5 – 8)** → start ≥ 5
- I7 (7 – 9) → overlaps with I5 → reject
- I6 (3 – 10) → overlaps → reject

Selected Activities

<i>I2, I</i>	<i>4, I</i>
--------------	-------------

These activities do not overlap and give the maximum number of compatible activities.

Time Complexity Analysis

- Sorting activities by finish time: **$O(n \log n)$**
- Selecting activities (single pass): **$O(n)$**

Total Time Complexity

$$O(n \log n)$$

Advantages of Greedy Activity Selection

- Simple and efficient approach
- Provides optimal solution
- Easy to implement
- Widely used in scheduling problems

OR

Q.3 (a) Prove that $\text{Log}(\sqrt{n}) = O(\log n)$. (3 mark)

Proof:

Big-Oh (O) notation represents the **upper bound** of a function and is used to describe the maximum growth rate of an algorithm.

To prove $\log(\sqrt{n}) = O(\log n)$, we first simplify the given expression using logarithmic properties.

$$\log(\sqrt{n}) = \log(n^{1/2})$$

Using the logarithm power rule:

$$\log(n^{1/2}) = \frac{1}{2} \log n$$

Thus, the function becomes a constant multiple of $\log n$.

According to Big-Oh definition, constant factors are ignored in asymptotic analysis.

So, there exists a constant $c = \frac{1}{2}$ and a value n_0 such that for all $n \geq n_0$:

$$\log(\sqrt{n}) \leq c \cdot \log n$$

Hence, it is proved that:

$$\log(\sqrt{n}) = O(\log n)$$

(b) Write any algorithm (need not to have meaningful) which has a time complexity of $O(1)$ and also includes a loop statement. Also, confirm its time complexity of $O(1)$ using tabular method. (4 mark)

Introduction

An algorithm is said to have **$O(1)$ time complexity** if the number of operations it performs is **constant** and does not depend on the input size n .

Even if a loop is present, the algorithm can still be $O(1)$ **provided the loop runs a fixed number of times**, independent of n .

Algorithm with Loop and $O(1)$ Time Complexity

Algorithm: ConstantTimeAlgorithm

- Step 1: Initialize a variable $\text{sum} = 0$
- Step 2: Execute a loop from $i = 1$ to 5
 - Add i to sum
- Step 3: Print the value of sum
- Step 4: Stop

Explanation:

The loop runs **exactly 5 times**, regardless of the input size. Therefore, the number of operations remains constant.

Pseudo Code

Algorithm ConstantTimeAlgorithm

```
1. sum ← 0
2. for i = 1 to 5 do
3.   sum ← sum + i
4. print(sum)
```

Tabular Method for Time Complexity Analysis

Step No.	Statement	Frequency	Time Taken
1	sum ← 0	1	c_1
2	for loop (i = 1 to 5)	5	c_2
3	sum ← sum + i	5	c_3
4	print(sum)	1	c_4

Total Time Calculation

$$T(n) = c_1 + 5c_2 + 5c_3 + c_4$$

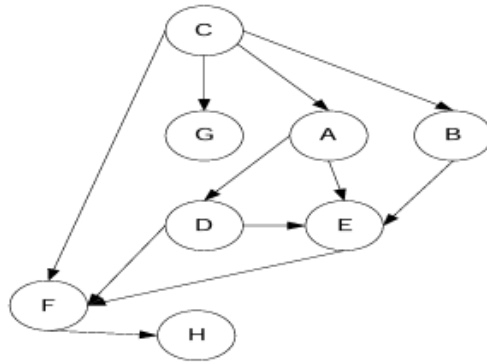
This is a **constant value**, as it does not depend on n .

Final Time Complexity

Ignoring constants as per asymptotic analysis:

$$\boxed{T(n) = O(1)}$$

(c) Write DFS traversal and draw DFS tree corresponds to following graph. Also, write time complexity of DFS algorithm. (7 mark)



Introduction / Definition

Depth First Search (DFS) is a **graph traversal algorithm** that explores a graph by going as deep as possible along one branch before backtracking. It uses the concept of **recursion or stack** and is applicable to both **directed and undirected graphs**. DFS visits each vertex exactly once and explores all outgoing edges of a vertex before moving to the next vertex.

Assumption for Traversal

- **Starting vertex: C**
 - When multiple adjacent vertices are available, they are visited in **alphabetical order**.
 - The given graph is a **directed graph**.
-

DFS Traversal of the Given Graph

Starting from vertex **C**, DFS proceeds as follows:

1. Start at **C**
2. From C, visit **A**
3. From A, visit **D**
4. From D, visit **E**

5. From E, visit **F**
6. From F, visit **H**
7. Backtrack to C and visit **B**
8. From C, finally visit **G**

DFS Traversal Order

$$C \rightarrow A \rightarrow D \rightarrow E \rightarrow F \rightarrow H \rightarrow B \rightarrow G$$

DFS Tree Description (Explained in Words)

- **C** is the root of the DFS tree.
- From C, the first child visited is **A**.
- From A, the next child is **D**.
- From D, DFS moves to **E**.
- From E, DFS continues to **F**.
- From F, DFS reaches **H**, which is a leaf node.
- After backtracking completely to C, DFS visits **B** as another child of C.
- Finally, **G** is visited as the last child of C.

Thus, the DFS tree shows a deep path

C → **A** → **D** → **E** → **F** → **H**,

with **B** and **G** as additional branches from C.

(If a diagram were drawn, it would show C at the top, a deep left-side chain, and B and G branching separately from C.)

Time Complexity of DFS

The time complexity of DFS depends on the representation of the graph:

- Using **Adjacency List**:

$$O(V + E)$$

- Using **Adjacency Matrix**:

$$O(V^2)$$

Where:

- V = number of vertices
 - E = number of edges
-

Applications of DFS

- Detecting cycles in graphs
 - Topological sorting
 - Finding connected components
 - Path finding and maze solving
-

Q.4 (a) What is Principle of Optimality in dynamic programming? Explain it with example. (3 mark)

Principle of Optimality – Definition

The **Principle of Optimality** is the fundamental concept of **Dynamic Programming**. It states that **an optimal solution to a problem contains optimal solutions to all of its subproblems**. In other words,

if a problem is solved optimally, then the decisions taken at each stage must also be optimal for the remaining subproblems.

Explanation

According to this principle, once a decision is made at a particular stage, the remaining problem becomes independent of the previous decisions except for the current state. Therefore, dynamic programming solves a problem by breaking it into smaller overlapping subproblems and solving each subproblem optimally only once. The optimal solution of the main problem is then constructed using these optimal subproblem solutions.

This principle justifies why dynamic programming works correctly and efficiently for optimization problems.

Example

Consider the **shortest path problem** in a graph. If the shortest path from node **A** to node **D** goes through node **B**, then the path from **A** to **B** must also be the shortest path between those two nodes. If it were not optimal, a better path to B would result in a shorter overall path to D, which contradicts the assumption that the original path was optimal.

(b) What is knapsack problem? Using greedy algorithm find an optimal solution for fractional knapsack instance $n=7$, $M=17$, profits $p[] = \{10, 5, 15, 7, 6, 16, 4\}$ and weights $w[] = \{2, 3, 5, 7, 1, 4, 1\}$. (4 mark)

Knapsack Problem – Definition

The **Knapsack Problem** is an optimization problem where we are given a set of items, each having a **profit** and a **weight**, and a knapsack with fixed **capacity M**. The objective is to **maximize total profit** without exceeding the knapsack capacity.

In the **Fractional Knapsack Problem**, items can be **divided into fractions**, and the greedy approach is used by selecting items based on **maximum profit-to-weight ratio**.

Given Data

- Number of items: $n = 7$
- Knapsack capacity: $M = 17$

Item Profit (p) Weight (w)

1	10	2
2	5	3
3	15	5
4	7	7
5	6	1
6	16	4
7	4	1

Step 1: Calculate Profit-to-Weight Ratio (p/w)

Item p w p/w

1	10	2	5
2	5	3	1.67
3	15	5	3
4	7	7	1

Item p w p/w

5 6 1 6

6 16 4 4

7 4 1 4

Step 2: Sort Items in Decreasing Order of p/w

Order of selection:

Item 5 → Item 1 → Item 6 → Item 7 → Item 3 → Item 2 → Item 4

Step 3: Fill the Knapsack Greedily

Item Weight Taken Profit Gained Remaining Capacity

Item 5 1 6 16

Item 1 2 10 14

Item 6 4 16 10

Item 7 1 4 9

Item 3 5 15 4

Item 2 3 5 1

Item 4 1/7 1 0

(Item 4 is taken fractionally)

Total Profit Calculation

$$\text{Total Profit} = 6 + 10 + 16 + 4 + 15 + 5 + 1 = \boxed{57}$$

(c) Find the optimal way of multiplying following matrices using dynamic programming. Also indicate optimal number of multiplications required. A: 2 x 3, B: 3 x 5, C: 5 x 6, D: 6 x 2, E: 2 x 3 (7 mark)

Introduction

The **Matrix Chain Multiplication problem** is a classic **Dynamic Programming** problem. The objective is to find the **most efficient order of multiplying a chain of matrices** such that the **total number of scalar multiplications is minimized**.

Matrix multiplication is associative, so the order of multiplication can be changed, but the **cost depends heavily on the chosen parenthesization**.

Dynamic programming is used because the problem exhibits **optimal substructure** and **overlapping subproblems**.

Step 1: Represent Matrix Dimensions

Let the dimension array be:

$$P = [2, 3, 5, 6, 2, 3]$$

Where:

- Matrix A = $P_0 \times P_1 = 2 \times 3$
 - Matrix B = $P_1 \times P_2 = 3 \times 5$
 - Matrix C = $P_2 \times P_3 = 5 \times 6$
 - Matrix D = $P_3 \times P_4 = 6 \times 2$
 - Matrix E = $P_4 \times P_5 = 2 \times 3$
-

Step 2: Dynamic Programming Formula

Let $M[i][j]$ be the **minimum number of scalar multiplications** required to multiply matrices from i to j .

$$M[i][j] = \min_{i \leq k < j} \{M[i][k] + M[k+1][j] + P_{i-1} \times P_k \times P_j\}$$

Step 3: Compute Costs of Different Parenthesizations

Length = 2 (Two matrices)

- **AB:** $2 \times 3 \times 5 = 30$
 - **BC:** $3 \times 5 \times 6 = 90$
 - **CD:** $5 \times 6 \times 2 = 60$
 - **DE:** $6 \times 2 \times 3 = 36$
-

Length = 3

- **(AB)C:** $30 + (2 \times 5 \times 6) = 90$
 - **A(BC):** $90 + (2 \times 3 \times 6) = 126 \rightarrow \mathbf{Min = 90}$
 - **(BC)D:** $90 + (3 \times 6 \times 2) = 126$
 - **B(CD):** $60 + (3 \times 5 \times 2) = 90 \rightarrow \mathbf{Min = 90}$
 - **(CD)E:** $60 + (5 \times 2 \times 3) = 90$
 - **C(DE):** $36 + (5 \times 6 \times 3) = 126 \rightarrow \mathbf{Min = 90}$
-

Length = 4

- **((AB)C)D:** $90 + (2 \times 6 \times 2) = 114$
- **(AB)(CD):** $30 + 60 + (2 \times 5 \times 2) = 110$
- **A(B(CD)):** $90 + (2 \times 3 \times 2) = 102 \rightarrow \mathbf{Min = 102}$
- **((BC)D)E:** $90 + (3 \times 2 \times 3) = 108$

- **(BC)(DE):** $90 + 36 + (3 \times 6 \times 3) = 180$
 - **B(C(DE)):** $90 + (3 \times 5 \times 3) = 135 \rightarrow \text{Min} = 108$
-

Length = 5 (Full Chain)

- **((AB)C)(DE):** $90 + 36 + (2 \times 6 \times 3) = 162$
 - **(A(B(CD)))E:** $102 + (2 \times 2 \times 3) = 114$
 - **A((BC)(DE)):** $108 + (2 \times 3 \times 3) = 126$
 - **(AB)((CD)E):** $30 + 90 + (2 \times 5 \times 3) = 150$
-

Optimal Solution

Minimum Number of Multiplications

114

Optimal Parenthesization

$((A(BC))D)E$

OR

Q.4 (a) What is the Travelling Salesman Problem (TSP)? Why TSP is considered an NP-hard problem? (3 mark)

Travelling Salesman Problem (TSP) – Definition

The **Travelling Salesman Problem (TSP)** is a well-known **combinatorial optimization problem**. In this problem, a salesman is given a list of **cities** and the **distance (or cost)** between every pair of cities. The objective is to find the **shortest possible tour** that starts

from a city, visits **each city exactly once**, and returns to the starting city.

TSP is widely used in fields such as routing, logistics, and circuit design.

Why TSP is NP-Hard

TSP is considered an **NP-hard problem** because:

- The number of possible tours increases **factorially** with the number of cities, making the problem computationally very expensive.
 - There is **no known polynomial-time algorithm** that can solve all instances of TSP optimally.
 - As the input size increases, the time required to find the exact solution grows **exponentially**.
 - TSP belongs to the class of problems for which **verifying a solution is easy**, but **finding the optimal solution is very difficult**.
-

(b) Illustrate the working of Kruskal's algorithm using greedy technique. Take the suitable graph for illustration. (4 mark)

Introduction / Definition

Kruskal's Algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph. A spanning tree connects all vertices with **minimum total edge weight** and without forming any cycles. The greedy strategy of Kruskal's algorithm is to **always select the edge with the minimum weight**, provided it does not form a cycle.

Working Principle of Kruskal's Algorithm

The algorithm works by considering edges in increasing order of their weights and gradually building the spanning tree.

Steps of Kruskal's Algorithm:

- Step 1: Sort all the edges of the graph in **ascending order of weight**
 - Step 2: Select the edge with the **smallest weight**
 - Step 3: Check whether adding this edge forms a **cycle**
 - If no cycle is formed, include the edge in MST
 - If a cycle is formed, discard the edge
 - Step 4: Repeat until the MST contains **$(V - 1)$ edges**, where V is number of vertices
-

Illustration Using a Suitable Graph

Consider a graph with vertices **A, B, C, D** and weighted edges:

- Edge $AB = 1$
- Edge $BC = 2$
- Edge $AC = 3$
- Edge $CD = 4$
- Edge $BD = 5$

Sorted edges by weight:

$AB (1), BC (2), AC (3), CD (4), BD (5)$

Selection Process:

- Select **AB (1)** → no cycle → include
- Select **BC (2)** → no cycle → include

- Select **AC (3)** → forms a cycle → discard
- Select **CD (4)** → no cycle → include

Now MST has **3 edges (V-1)**, so the algorithm stops.

(If drawn, the diagram would show nodes connected by the lowest-weight edges without forming a loop.)

Key Features of Kruskal's Algorithm

- Uses **greedy approach**
- Builds MST edge by edge
- Uses **cycle detection** (often using Union-Find)
- Suitable for **sparse graphs**

(c) Find the longest common subsequence for the following two sequences using dynamic programming. Show the complete process and indicate the time complexity. $X = 10101010011$ $Y = 101001$ (7 mark)

Introduction / Definition

The **Longest Common Subsequence (LCS)** problem is a classical **Dynamic Programming** problem. A subsequence is a sequence that can be derived from another sequence by deleting zero or more characters **without changing the relative order** of the remaining characters.

The objective of LCS is to find the **maximum length subsequence** that is common to both given sequences.

Dynamic programming is used because the problem has **optimal substructure** and **overlapping subproblems**.

Step 1: Define DP Table

Let:

- $X = x_1x_2 \dots x_m$ where $m = 11$
- $Y = y_1y_2 \dots y_n$ where $n = 6$

Define a DP table $L[i][j]$ which represents the **length of LCS** of the prefixes:

- $X[1..i]$ and $Y[1..j]$
-

Step 2: LCS Recurrence Relation

The recurrence relation for LCS is:

- If $x_i = y_j$:

$$L[i][j] = 1 + L[i - 1][j - 1]$$

- If $x_i \neq y_j$:

$$L[i][j] = \max(L[i - 1][j], L[i][j - 1])$$

Boundary condition:

$$L[i][0] = L[0][j] = 0$$

Step 3: Fill the DP Table (Explanation in Words)

- Start filling the table row by row.
- Compare each character of sequence **X** with sequence **Y**.
- Whenever characters match, increase the LCS length by 1 from the diagonal value.

- When they do not match, take the maximum value from the top or left cell.

After completing the table, the value at **L[11][6]** gives the length of the LCS.

Step 4: Length of LCS

After filling the DP table completely:

$$\boxed{\text{Length of LCS} = 5}$$

Step 5: Constructing the LCS

By tracing back from **L[11][6]** to **L[0][0]**:

- Move diagonally when characters match
- Move to the cell with larger value when characters do not match

One possible **Longest Common Subsequence** is:

10101

(This subsequence appears in both X and Y in the same relative order.)

Explanation of Result

- The subsequence **10101** can be formed from X by deleting some characters.
- The same subsequence appears in Y without changing order.
- No longer common subsequence exists between X and Y.

Time Complexity Analysis

- Filling the DP table takes $m \times n$ operations
- Here, $m = 11$ and $n = 6$

Time Complexity

$$O(mn)$$

Space Complexity

$$O(mn)$$

Q.5 (a) Define P and NP problems. Also give example of each type of problem. (3 mark)

P Problems – Definition

P (Polynomial time) problems are the class of decision problems that can be **solved in polynomial time** by a deterministic Turing machine. In simple words, these are problems for which an algorithm exists that can find the solution efficiently, and the time required grows as a polynomial function of the input size.

Examples of P Problems:

- Linear search
 - Binary search
 - Sorting algorithms like Merge Sort and Heap Sort
 - Shortest path problem using Dijkstra's algorithm (with non-negative weights)
-

NP Problems – Definition

NP (Nondeterministic Polynomial time) problems are the class of decision problems for which a given solution can be **verified in polynomial time**, even if finding the solution may take exponential time. These problems are solved by a nondeterministic Turing machine in polynomial time.

Examples of NP Problems:

- Travelling Salesman Problem (decision version)
- Boolean Satisfiability Problem (SAT)
- Hamiltonian Cycle Problem
- Subset Sum Problem

(b) Explain the backtracking strategy to solve optimization problem. Explain how it can be applied to find the solution for 4-queen problem. (3 mark)

Backtracking Strategy – Definition

Backtracking is a systematic algorithmic technique used to solve **optimization and decision problems** by exploring all possible solutions **step by step**. It builds the solution incrementally and **abandons (backtracks)** a partial solution as soon as it is determined that the partial solution cannot lead to an optimal or valid solution.

This strategy reduces unnecessary computation by **pruning the search space** and is widely used for combinatorial problems.

Backtracking for Optimization Problems

In optimization problems, backtracking:

- Explores all possible choices

- Checks feasibility at each step
- Rejects solutions that violate constraints
- Continues searching until the **best (optimal) solution** is found

It ensures correctness by considering all valid possibilities while avoiding invalid paths early.

Application to 4-Queen Problem

Problem Description

The **4-Queen Problem** is to place **4 queens** on a **4×4 chessboard** such that:

- No two queens attack each other
 - No two queens share the same **row, column, or diagonal**
-

Backtracking Approach for 4-Queen Problem

- Place one queen in the **first row**
- Move to the next row and place a queen in a **safe column**
- Check constraints:
 - Same column
 - Same diagonal
- If a conflict occurs:
 - **Backtrack** and try the next possible position
- Continue this process until all 4 queens are placed safely

If no valid position is available in a row, the algorithm backtracks to the previous row and changes the queen's position.

(If a diagram were drawn, it would show queens placed row by row, with backtracking when conflicts arise.)

(c) Given a text and a pattern, write the naive string matching algorithm to find all occurrences of the pattern in the text. Use the text "ABCABCABC" and the pattern "ABC". Also, give time complexity of naive string matching algorithm. (7 mark)

Naive String Matching Algorithm

Introduction / Definition:

String matching is the process of finding occurrences of a pattern string P in a text string T . The **naive string matching algorithm** is the simplest method, which checks for the pattern at every possible position in the text. It does not use any preprocessing or advanced data structures, making it straightforward but less efficient for large texts.

Detailed Explanation

The naive string matching algorithm works as follows:

1. Let the text be T of length n and the pattern be P of length m .
2. Start from the first character of T and compare it with the first character of P .
3. If the first character matches, continue comparing subsequent characters of P with the corresponding characters in T .
4. If all characters of P match consecutively in T , record this position as an occurrence.
5. Move the pattern one position forward in T and repeat the process until the end of the text is reached.

This approach checks for the pattern at every possible starting index of the text.

Steps / Procedure

1. Initialize:

- Text $T = \text{"ABCABCABC"}$
- Pattern $P = \text{"ABC"}$
- Text length $n = 9$, Pattern length $m = 3$

2. Algorithm Pseudocode:

NaiveStringMatching(T, P):

```
n = length(T)
m = length(P)
for i from 0 to n - m:
    j = 0
    while j < m and T[i + j] == P[j]:
        j = j + 1
    if j == m:
        print("Pattern found at index", i)
```

3. Execution on Given Example:

- Check index 0: $T[0..2] = \text{"ABC"}$ matches $P = \text{"ABC"}$ → Pattern found at index 0
- Check index 1: $T[1..3] = \text{"BCA"}$ → No match
- Check index 2: $T[2..4] = \text{"CAB"}$ → No match
- Check index 3: $T[3..5] = \text{"ABC"}$ → Pattern found at index 3
- Check index 4: $T[4..6] = \text{"BCA"}$ → No match
- Check index 5: $T[5..7] = \text{"CAB"}$ → No match

- Check index 6: $T[6..8] = \text{"ABC"} \rightarrow$ Pattern found at index 6

Result: Pattern occurs at indices **0, 3, and 6**.

Key Features

- Simple and easy to implement
 - No preprocessing of text or pattern needed
 - Checks every possible alignment
-

Advantages

- Works for all types of text and patterns
- Easy to understand and code

Disadvantages

- Inefficient for large texts or patterns
 - Time complexity is high in worst case
-

Time Complexity

- **Worst-case time complexity:** $O((n - m + 1) \cdot m) \approx O(n \cdot m)$
 - Occurs when all characters match except the last one at each shift
 - **Best-case time complexity:** $O(n)$
 - When mismatches occur at the first character of the pattern
-

Applications

- Searching keywords in documents

- Text editors and IDEs for find/replace functionality
 - Pattern searching in DNA sequences
-

OR

Q.5 (a) What is a polynomial reduction? Explain its role in proving NP completeness. (3 mark)

Polynomial Reduction

Definition / Introduction:

Polynomial reduction is a technique used in computational complexity theory to relate the difficulty of two problems. A problem A is said to be **polynomially reducible** to a problem B (denoted as $A \leq_p B$) if there exists a **polynomial-time algorithm** that transforms any instance of A into an instance of B such that the solution of B gives the solution for A . Essentially, it shows that solving B efficiently would also solve A efficiently.

Key Points:

- It is used to compare the computational difficulty of problems.
 - The transformation from problem A to problem B must be done in **polynomial time**.
 - If $A \leq_p B$ and B is solvable in polynomial time, then A is also solvable in polynomial time.
 - It is a fundamental tool in **NP-completeness proofs**.
-

Role in Proving NP-Completeness:

- To prove a problem X is NP-complete, we need to show:

1. $X \in NP$ (the solution can be verified in polynomial time).
 2. An already known NP-complete problem Y can be **polynomially reduced** to X .
- This proves that X is **at least as hard as** Y .
 - Polynomial reduction helps in **building a chain of reductions** from known NP-complete problems to new problems, thereby establishing their NP-completeness.
-

Example (Simple Explanation):

- Suppose **3-SAT** is NP-complete. If we can reduce 3-SAT to a new problem **Subset Sum** in polynomial time, then Subset Sum is also NP-complete.

(b) Give example of any NP complete problem. Explain how to prove that particular problem is NP complete. (4 mark)

NP-Complete Problem Example and Proof

Definition / Introduction:

An **NP-complete problem** is a problem that is both in **NP** (its solution can be verified in polynomial time) and **NP-hard** (every problem in NP can be reduced to it in polynomial time). NP-complete problems are considered the most difficult problems in NP, and solving any one of them efficiently would solve all problems in NP efficiently.

Example: 3-Satisfiability Problem (3-SAT)

- **Problem Statement:**
Given a Boolean formula in **conjunctive normal form (CNF)** where each clause has exactly three literals, determine whether

there exists an assignment of truth values to variables that makes the formula true.

- **Explanation:**

For example, consider the formula:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (x_2 \vee x_3 \vee \neg x_4)$$

We need to find whether there is a combination of true/false values for x_1, x_2, x_3, x_4 that satisfies all clauses simultaneously.

Steps to Prove 3-SAT is NP-Complete

1. Show 3-SAT $\in$ NP:

- If a candidate assignment of variables is given, we can **verify** in polynomial time whether it satisfies all clauses.
- Checking each clause for satisfaction requires a fixed number of steps; for m clauses, total verification is polynomial in m .

2. Polynomial-time Reduction from Known NP-Complete Problem:

- Use a known NP-complete problem such as **SAT (Satisfiability Problem)**.
- Any instance of SAT can be converted into an equivalent 3-SAT instance in polynomial time.
- This ensures 3-SAT is at least as hard as SAT.

3. Conclusion:

- Since 3-SAT is in NP and any NP problem can be reduced to it in polynomial time, **3-SAT is NP-complete**.

Key Features:

- Verification is polynomial-time.
 - Reduction from other NP problems is possible.
 - Widely used in complexity theory to prove NP-completeness of other problems.
-

Applications:

- Circuit design and verification
- Scheduling problems
- Cryptography and security

(c) Show that Hamiltonian cycle is a NP problem. (7 mark)

Introduction / Definition:

A **Hamiltonian cycle** in a graph is a cycle that visits each vertex **exactly once** and returns to the starting vertex. The **Hamiltonian Cycle Problem (HCP)** asks whether a given graph contains such a cycle. This problem is a classical example in computational complexity theory and plays a significant role in the study of NP-complete problems.

Detailed Explanation

1. Problem Statement:

Given a graph $G = (V, E)$, where V is the set of vertices and E is the set of edges, determine if there exists a cycle that visits every vertex **exactly once** and ends at the starting vertex.

2. Understanding the Problem:

- The cycle must cover all vertices.
- No vertex can be visited more than once.

- The graph can be **directed** or **undirected**.
- The problem is decision-based: answer is **Yes** or **No**.

3. Why Hamiltonian Cycle is an NP Problem:

To show that HCP belongs to NP, we need to prove that **any proposed solution (certificate) can be verified in polynomial time**:

Steps to Verify a Hamiltonian Cycle:

- Suppose we are given a sequence of vertices $C = (v_1, v_2, \dots, v_n, v_1)$ claiming to be a Hamiltonian cycle.
- **Step 1:** Check that every vertex in the graph appears **exactly once** in the sequence except the first vertex, which repeats at the end.
- **Step 2:** For each consecutive pair of vertices (v_i, v_{i+1}) , check if an **edge exists** in the graph.
- **Step 3:** Verify that the last vertex connects back to the first vertex.

If all these checks pass, the cycle is valid.

4. Time Complexity of Verification:

- Checking the sequence length and uniqueness of vertices takes $O(n)$.
- Checking edges between consecutive vertices takes $O(n)$.
- Total verification time is $O(n)$, which is **polynomial in the number of vertices**.

Key Features

- Decision problem: Yes/No answer

- Input: Graph with n vertices
 - Certificate: A sequence of vertices forming a Hamiltonian cycle
 - Verification: Polynomial-time check
-

Advantages of Studying Hamiltonian Cycle

- Helps in understanding NP and NP-complete problems
 - Provides insight into graph theory and combinatorial optimization
 - Forms a basis for reductions to prove other NP-complete problems
-

Disadvantages / Challenges

- Finding a Hamiltonian cycle is **computationally hard** (no known polynomial-time algorithm for general graphs)
 - Exponential number of possible sequences ($n!$) makes brute-force search impractical for large graphs
-

Applications

- **Traveling Salesman Problem (TSP):** Finding the shortest Hamiltonian cycle in weighted graphs
 - **Network routing:** Ensuring all nodes are visited exactly once
 - **Scheduling problems:** Ordering tasks or jobs without repetition
-

Example Explanation

- Consider a graph with vertices $V = \{A, B, C, D\}$ and edges connecting them in a square:

- Edges: $(A, B), (B, C), (C, D), (D, A), (A, C), (B, D)$

- A proposed Hamiltonian cycle: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

- **Verification Steps:**

1. All vertices appear once $\rightarrow \checkmark$
2. All consecutive edges exist $\rightarrow \checkmark$
3. Last vertex connects to first $\rightarrow \checkmark$

- Hence, the sequence is a valid Hamiltonian cycle.

A diagram would show a square with vertices A, B, C, D connected, highlighting the cycle.
